

第 9 讲

2D Tiled Convolution Kernel; Reuse Analysis

目录

1 核心概念与分块策略	1
1.1 卷积与模板计算	1
1.2 三种分块策略	2
1.3 课程事项	3
2 二维分块卷积的线程组织与实现	3
2.1 输入范围与网格步长	3
2.2 三组坐标的对应关系	4
2.3 协作加载、补零与同步	5
2.4 输出计算与写回条件	5
2.5 二维策略 3 的对应变化	6
2.6 完整内核与主机启动	6
3 数据复用与全局访存计数	9
3.1 复用因子的计数口径	9
3.2 一维内部块：按输出与按输入两种计数	9
3.3 一维边界块：虚拟元素不产生全局读取	11
3.4 二维内部块：两个方向的复用相乘	12
3.5 二维边界的乘积计数	12
4 带宽、算术强度与计算上限	14
4.1 未分块卷积的字节 / 浮点操作比	14
4.2 由内存带宽得到吞吐上限	14
4.3 不同参数下的复用门槛	15
4.4 权重窗口大小与资源平衡	15
4.5 扩展计数：将输出写回计入流量	16
5 分块规模与存储层次	17
5.1 线程数量与共享内存容量	17
5.2 更大数据块的多元素分工	18
5.3 分银行存储与访问组织	19
6 例题与关键结论	20
6.1 角块的全局读取缩减倍数	20
6.2 输入型线程块中的控制分歧	21
6.3 A40 算例中的带宽平衡与复用门槛	21
6.4 关键公式与实现关系	22

1 核心概念与分块策略

1.1 卷积与模板计算

模板算法 (stencil algorithm) 按照固定的邻域模式更新数组元素。邻域规定一个输出依赖哪些输入；计算规则规定这些输入如何组合。二维卷积是其中一种：每个输出由一个二维邻域内的输入与相应权重相乘，再求和得到。其他模板可以只使用少量特定方向的邻点，例如二维五点模板、三维十三点有限差分模板，以及四维 Wilson-Dslash 模板。分块所利用的共同性质，是相邻输出的输入邻域存在重叠。

设输入数组为 N ，输出数组为 P ，两者均有 H 行、 W 列；权重矩阵为 M ，边长为奇数 m ，半径为 $r = (m - 1)/2$ 。用 $\tilde{N}$ 表示在数组范围外补零后的输入。为明确后续代码的下标关系，按其权重访问顺序写出计算式：

$$P[y, x] = \sum_{i=0}^{m-1} \sum_{j=0}^{m-1} M[i, j] \tilde{N}[y - r + i, x - r + j]. \quad (1)$$

这里沿用权重按 $M[i, j]$ 直接访问的约定；公式的用途是说明计算邻域与数组下标。权重数组在代码中记为 $\mathbf{Mc}$ 。例如 $m = 5$ 时，每个内部输出需要一个 5×5 输入窗口，共进行 25 次乘法和 25 次累加。

输出分块 (output tile) 是一组由同一线程块负责生成的相邻输出。输入分块 (input tile) 则包含计算这些输出所需的全部输入，其中超出输出分块对应核心区域的部分称为光环区 (halo)。落到整个输入数组之外、由边界规则补出的值称为虚拟边界元素 (ghost element)。光环描述块与块之间的邻域关系；虚拟边界元素描述整个数组以外的位置。内部块也有光环，但其光环通常仍是有效数组元素。

表 1 二维方形分块的统一记号

记号	含义	代码中的对应量
H, W	输入和输出的行数、列数	Height, Width
m, r	权重边长及半径, $m = 2r + 1$	MASK_WIDTH, RADIUS
T	输出分块的边长	TILE_WIDTH
S	输入分块的边长, $S = T + m - 1$	INPUT_WIDTH
t_x, t_y	线程块内的二维线程坐标	threadIdx.x, threadIdx.y
b_x, b_y	网格内的二维线程块坐标	blockIdx.x, blockIdx.y

本讲的学习目标是理解分块卷积算法中的输入分块与输出分块，进一步分析卷积和模板计算中的数据复用，并为 Lab 4 做准备。

1.2 三种分块策略

线程可以围绕输出计算分工，也可以围绕输入加载分工。输出分工使一个线程自然地对应一个输出元素；加载分工则使一个线程对应一个需要装入共享内存的输入元素。由于输入分块通常更大，这两种分工对应不同的线程块大小。

策略 1：线程块覆盖输出分块。用 T^2 个线程生成 T^2 个输出，但需要准备 S^2 个输入。因此部分线程要通过多次加载搬入光环元素。所有线程在计算阶段都可以承担输出，加载阶段则需要额外的索引安排。

策略 2：线程块覆盖输入分块。用 S^2 个线程分别写入共享内存中的一个位置，再由其中的 T^2 个线程计算输出。加载工作统一，计算阶段则有一部分线程不再执行乘加循环。这是后续二维内核的主要组织方式。

策略 3：共享内存只保存核心区。用 T^2 个线程加载与输出分块同样大小的核心输入区域。计算邻域落在核心区内时读取共享内存；落在光环区时读取全局内存。因此乘加循环内部需要判断数据位置，且光环仍可能产生重复的全局读取。

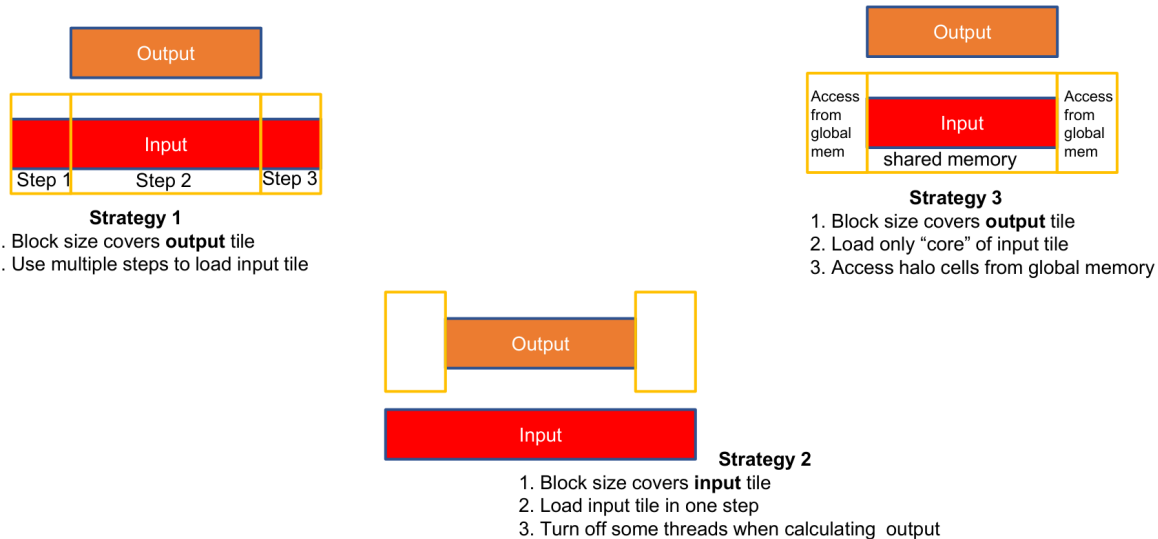


图 1 三种策略的加载范围与线程分工。策略 1 分多步装入完整输入，策略 2 为完整输入分配线程，策略 3 仅将核心区置于共享内存。

策略 2 的取舍是把更整齐的工作分配用于高延迟的输入加载，并把线程参与程度的差异留给乘加计算。它避免为了补光环而单独组织很窄的一段访问。这里“加载一致”针对块内各线程的职责；靠近整幅图像边界时，有效输入与补零位置仍需分别处理。

1.3 课程事项

表 2 2026 年 9 月 22 日公布的课程安排

事项	安排
实验与项目	Lab 3 于当周星期五截止；项目里程碑 1 于当周发布。CNN 为个人项目，GPT-2 为 3-4 人团队项目。
第一次期中考试	10 月 6 日 19:00-22:00，纸笔考试；范围为 Lectures 1-11 和 Labs 1-4。具体教室由 Canvas 公布。
考试时间冲突 复习与阅读	有有效时间冲突者，须在 9 月 30 日前发邮件联系教师，另行安排考试时间。 复习资料已在 Canvas 发布；阅读教材第 7 章，并提交 Lab 3。

2 二维分块卷积的线程组织与实现

2.1 输入范围与网格步长

一个边长为 T 的输出分块，在每个方向上都需要额外的左右或上下邻域。以列方向为例，输出列范围为 $b_x T$ 到 $b_x T + T - 1$ ；最左侧输出需要向左延伸 r 列，最右侧输出需要向右延伸 r 列，因此输入列范围为 $b_x T - r$ 到 $b_x T + T - 1 + r$ 。含首尾两端，共有

$$S = (b_x T + T - 1 + r) - (b_x T - r) + 1 = T + 2r = T + m - 1 \quad (2)$$

列。行方向完全相同，故完整输入分块是 $S \times S$ 。当 $m = 5$ 时，每侧增加 2 个元素，输入边长为 $T + 4$ 。

策略 2 的线程块大小为 $(S, S, 1)$ ，每个线程负责输入分块中的一个位置。网格大小则由输出覆盖范围决定：

$$\text{gridDim.x} = \left\lceil \frac{W}{T} \right\rceil, \quad \text{gridDim.y} = \left\lceil \frac{H}{T} \right\rceil. \quad (3)$$

相邻块的输出起点相差 T ，恰好使输出分块相接；输入起点也相差 T ，因此相邻输入分块重叠 $S - T = m - 1$ 列或行。若用 S 作为输出步长，两个相邻输出分块之间会留下宽度为 $m - 1$ 的缺口。

例如 $H = 37, W = 53, T = 16, m = 5$ ，应建立 4×3 个线程块，每块有 $20 \times 20 = 400$ 个线程。右下角块的输出起点为 $(32, 48)$ ，只有 5×5 个输出落在图像内，但该块仍按完整的输入分块组织加载与同步。

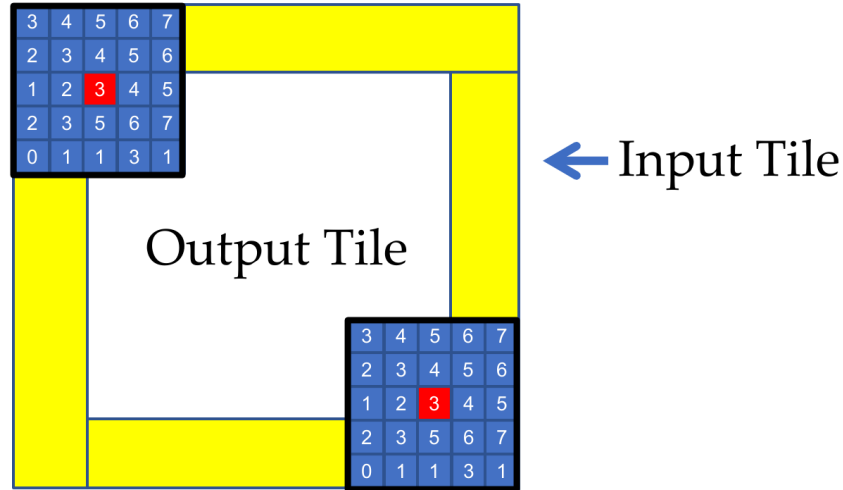


图2 计算输出分块两个相对角点时，其输入窗口分别延伸到左上与右下的光环区域。白色输出区域外的黄色边框也是输入分块的一部分。

2.2 三组坐标的对应关系

线程局部坐标 (t_y, t_x) 同时参与输出坐标与输入加载坐标的构造，两者之间相差半径 r ：

$$\begin{aligned} y_o &= b_y T + t_y, & x_o &= b_x T + t_x, \\ y_i &= y_o - r, & x_i &= x_o - r. \end{aligned} \quad (4)$$

每个线程把 $N[y_i, x_i]$ 放入共享内存的 `tile[ty][tx]`。当 $t_y < T$ 且 $t_x < T$ 时，它还负责输出 $P[y_o, x_o]$ 。其余线程虽然可以算出形式上的 (y_o, x_o) ，但这些坐标并不代表本块实际负责的输出。

共享内存中的位置 (s_y, s_x) 对应的全局输入坐标为

$$(b_y T - r + s_y, b_x T - r + s_x). \quad (5)$$

将 $s_y = t_y + i, s_x = t_x + j$ 代入，可得 `tile[t_y + i][t_x + j]` 恰好对应 $\tilde{N}[y_o - r + i, x_o - r + j]$ 。于是卷积窗口可直接从共享内存局部坐标 (t_y, t_x) 开始访问，不再在内层循环里减半径。

加载的元素与输出的中心

线程 (t_y, t_x) 加载的是其输出窗口的左上角输入。输出中心对应的输入位于共享内存 $(t_y + r, t_x + r)$ ，通常由另一个线程加载。因此参加输出计算的是线程坐标中的左上 $T \times T$ 区域；无需把计算条件改成 $r \leq t_x, t_y < r + T$ 。

取 $T = 8, m = 5$ 和块 $(b_x, b_y) = (1, 2)$ 。输入分块覆盖行 14–25、列 6–17；输出分块覆盖行

16-23、列 8-15。线程 $(t_y, t_x) = (0, 0)$ 加载 $N[14, 6]$ ，计算 $P[16, 8]$ ；线程 $(7, 7)$ 加载 $N[21, 13]$ ，计算 $P[23, 15]$ ；线程 $(11, 11)$ 加载 $N[25, 17]$ ，只参与加载。最后这个输入正是右下角输出窗口需要的元素。

2.3 协作加载、补零与同步

所有 S^2 个线程都向共享内存写入一个值。若 $0 \leq y_i < H$ 且 $0 \leq x_i < W$ ，就读取有效输入；否则写入 `0.0f`。这样，共享内存中的整个输入分块都已初始化，乘加循环可以统一访问一个完整窗口。

输入越界与输出越界要分别判断。左上角线程可能加载一个负坐标位置，却仍要计算有效输出。例如第一块的线程 $(0, 0)$ 加载输入坐标 $(-r, -r)$ ，需要写入零，但它负责的输出坐标是 $(0, 0)$ 。反过来，右边缘某个线程的输出坐标可能超过 $W - 1$ ，而其左移 r 后的输入坐标仍然有效，且可能被附近的有效输出使用。

加载完成后执行 `__syncthreads()`。其位置必须在所有线程的加载或补零之后，在任何线程读取他人写入的共享数据之前。由于一个输出窗口包含多个线程写入的元素，单个线程完成自己的写入，并不表示整个窗口都已经准备好。

两个判断、一个公共同步点

先根据输入坐标决定“读取输入”或“写入零”，随后让整个线程块经过同步点，最后根据线程角色及输出边界决定是否计算和写回。不能在同步之前仅因某个线程没有有效输出而提前退出，否则既可能漏掉其他线程需要的加载，也会破坏这一统一的协作结构。

2.4 输出计算与写回条件

参加计算的线程满足 $t_x < T$ 且 $t_y < T$ 。它将累加器初始化为零，执行 $m \times m$ 次乘加，然后只在 $y_o < H$ 且 $x_o < W$ 时写回。行优先存储（row-major storage）中，二维位置 (y, x) 的线性偏移是 $yW + x$ 。

共享内存访问的最大行下标为 $(T - 1) + (m - 1) = S - 1$ ，列下标也相同。因此，只要加载分块确实分配为 $S \times S$ ，上述计算范围与共享内存范围就一致。这里的边界检查分别承担不同任务：线程坐标限制本块拥有的输出，输出坐标限制整幅图像内的写入位置。

矩形图像的边界条件。行坐标应与高度比较，即 `row_o < Height`；列坐标应与宽度比较，即 `col_o < Width`。混用高度与宽度会使矩形图像出现漏写或越界。权重边长由编译期常量 `MASK_WIDTH` 指定，两个循环均按这一边长遍历。

2.5 二维策略 3 的对应变化

策略 3 将共享内存缩为 $T \times T$ ，线程 (t_y, t_x) 加载其输出中心对应的核心输入。此时权重位置 (i, j) 需要访问的核心区局部坐标为 $(t_y - r + i, t_x - r + j)$ 。若两个局部坐标均落在 $[0, T)$ ，从共享内存读取；否则根据全局坐标读取光环，越过整幅图像边界时按零处理。

因此，策略 2 在加载时一次性解决输入分块边界，乘加循环统一读取共享内存；策略 3 在每次乘加时判断目标属于核心区还是光环区。后者不需要仅负责光环加载的线程，但同一线程束（warp）中的线程可能在内层循环选择不同的读取分支。这一差异同时影响全局访问次数和控制流，后续复用公式要与具体的加载方式对应。

2.6 完整内核与主机启动

下面将加载、同步、计算与写回连成一个完整内核。示例取 $T = 16, m = 5$ ，每块 400 个线程，分配 400 个单精度共享内存元素；其中 256 个线程具有输出角色。Mc 在启动前由主机填入，输入与输出使用互不重叠的设备缓冲区。

代码 1 二维策略 2: 完整输入分块的协作加载与输出计算

```

1  #pragma once
2  #include <cuda_runtime.h>
3
4  constexpr int TILE_WIDTH = 16;
5  constexpr int MASK_WIDTH = 5;
6  constexpr int RADIUS = MASK_WIDTH / 2;
7  constexpr int INPUT_WIDTH = TILE_WIDTH + MASK_WIDTH - 1;
8  static_assert(TILE_WIDTH > 0 && MASK_WIDTH % 2 == 1);
9
10 __constant__ float Mc[MASK_WIDTH][MASK_WIDTH];
11
12 __global__ void convolution2DStrategy2(
13     const float* N, float* P, int Height, int Width)
14 {
15     __shared__ float tile[INPUT_WIDTH][INPUT_WIDTH];
16     const int tx = threadIdx.x;
17     const int ty = threadIdx.y;
18     const int row_o = int(blockIdx.y) * TILE_WIDTH + ty;
19     const int col_o = int(blockIdx.x) * TILE_WIDTH + tx;
20     const int row_i = row_o - RADIUS;
21     const int col_i = col_o - RADIUS;
22
23     if (row_i >= 0 && row_i < Height &&
24         col_i >= 0 && col_i < Width) {
25         tile[ty][tx] = N[size_t(row_i) * Width + col_i];
26     } else {
27         tile[ty][tx] = 0.0f;
28     }
29     __syncthreads();
30
31     if (ty < TILE_WIDTH && tx < TILE_WIDTH) {
32         float Pvalue = 0.0f;
33         for (int i = 0; i < MASK_WIDTH; ++i) {
34             for (int j = 0; j < MASK_WIDTH; ++j) {
35                 Pvalue += Mc[i][j] * tile[ty + i][tx + j];
36             }
37         }
38         if (row_o < Height && col_o < Width) {
39             P[size_t(row_o) * Width + col_o] = Pvalue;
40         }
41     }
42 }

```

主机端的关键区别仍是“块尺寸用 S ，网格覆盖用 T ”。以下接口负责权重复制、参数检查和内核启动，并返回相应错误状态。 h_Mc 指向行优先的 $m \times m$ 主机权重数组， d_N 与 d_P 已分别分配至少 HW 个单精度元素。

代码 2 主机端的启动接口与设备资源检查

```

1 #include "convolution2d_strategy2.cuh"
2
3 cudaError_t launchConvolution(const float* d_N, float* d_P,
4                               const float* h_Mc,
5                               int Height, int Width)
6 {
7     if (!d_N || !d_P || !h_Mc || d_N == d_P ||
8         Height <= 0 || Width <= 0)
9         return cudaErrorInvalidValue;
10
11     int device = 0;
12     cudaError_t err = cudaGetDevice(&device);
13     if (err != cudaSuccess) return err;
14     cudaDeviceProp prop;
15     err = cudaGetDeviceProperties(&prop, device);
16     if (err != cudaSuccess) return err;
17     if (INPUT_WIDTH * INPUT_WIDTH > prop.maxThreadsPerBlock ||
18         INPUT_WIDTH > prop.maxThreadsDim[0] ||
19         INPUT_WIDTH > prop.maxThreadsDim[1] ||
20         sizeof(float) * INPUT_WIDTH * INPUT_WIDTH >
21         prop.sharedMemPerBlock)
22         return cudaErrorInvalidConfiguration;
23
24     err = cudaMemcpyToSymbol(Mc, h_Mc, sizeof(Mc));
25     if (err != cudaSuccess) return err;
26     dim3 block(INPUT_WIDTH, INPUT_WIDTH, 1);
27     dim3 grid((Width - 1) / TILE_WIDTH + 1,
28              (Height - 1) / TILE_WIDTH + 1, 1);
29     if (grid.x > unsigned(prop.maxGridSize[0]) ||
30         grid.y > unsigned(prop.maxGridSize[1]))
31         return cudaErrorInvalidConfiguration;
32     convolution2DStrategy2<<<grid, block>>>(d_N, d_P, Height, Width);
33     return cudaGetLastError();
34 }

```

此接口返回的是参数检查、权重复制或内核启动阶段的错误。调用方需要在取得结果前完成相应的设备同步或数据传回，并处理执行阶段可能返回的错误。代码中的资源检查也说明， S 不能仅由复用公式任意增大：线程块还必须满足实际设备的单块线程数、各维大小及共享内存容量限制。

3 数据复用与全局访存计数

3.1 复用因子的计数口径

相邻输出反复使用同一个输入元素。直接实现每次需要它时都从全局内存读取；完整分块实现先把它读入共享内存，再为多个输出提供数据。设直接实现对输入数组的读取次数为 A ，分块后为准备这些输出而需要加载的不同有效输入数为 B ，定义

$$R = \frac{A}{B}. \quad (6)$$

R 是输入全局读取量的缩减倍数，也是一次输入加载平均支持的有效使用次数。若 $R = 4$ ，分块后的这部分全局读取量是原来的四分之一。权重读取、输出写回以及访问被硬件合并或缓存后的物理事务量，不包含在这一计数口径中。

分块几何给出的是可利用的复用量。对于把完整输入分块逐元素装入共享内存的策略 1 和策略 2， B 可以直接取有效输入元素数；对于只保存核心区的策略 3，还要计入计算期间发生的光环全局读取。因而同一个几何复用公式，不能直接作为三种实现实际全局流量都相等的依据。

3.2 一维内部块：按输出与按输入两种计数

3.2.1 从输出需要多少输入开始

取 $T = 8, m = 5$ ，一个内部输出块为 $P[8]$ 到 $P[15]$ 。 $P[8]$ 需要 $N[6]$ 到 $N[10]$ ， $P[9]$ 需要 $N[7]$ 到 $N[11]$ ，依次移动后， $P[15]$ 需要 $N[13]$ 到 $N[17]$ 。全部输出总共使用 $8 \times 5 = 40$ 个输入值，但不同的输入元素只有 $N[6]$ 到 $N[17]$ 这 12 个。

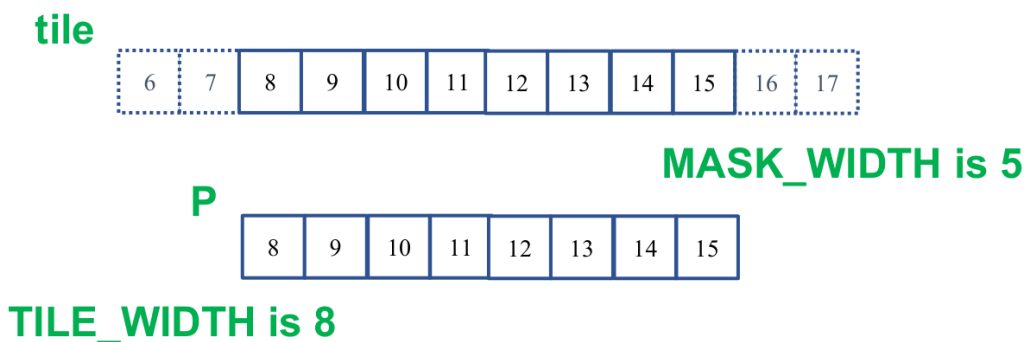


图 3 8 个连续输出对应 12 个不同输入。左右各两个光环元素保证最外侧输出也能获得完整的五点窗口。

因此，40 次直接全局读取可由 12 次输入加载加共享内存读取完成， $R = 40/12 = 10/3 \approx 3.3$ 。若每个输入为 4 字节，这部分全局读入数据由 160 字节降至 48 字节。对于一般的内部块，输出共有 T 个，每个需要 m 个输入，不同输入数为 $T + m - 1$ ，于是

$$R_{ID}(T, m) = \frac{Tm}{T + m - 1}. \quad (7)$$

只要块完全位于输入内部，该比值由 T 和 m 决定，与整条输入数组的总长度无关。

3.2.2 从每个输入被使用几次开始

同一例子也可以按输入计数。 $N[6]$ 仅供 $P[8]$ 使用， $N[7]$ 供两个输出使用，靠近中间的输入被五个输出使用，另一侧再逐渐减少。按 $N[6]$ 到 $N[17]$ 的顺序，使用次数为

$$1, 2, 3, 4, 5, 5, 5, 5, 4, 3, 2, 1.$$

这些次数之和仍为 40。光环附近的低复用输入拉低了平均值，所以有限分块的平均复用小于单个中央输入能达到的五次使用。

当 $T \geq m$ 时，两侧各形成一个从 1 到 $m - 1$ 的阶梯，中间有 $T - m + 1$ 个输入各使用 m 次。由此得到

$$\begin{aligned} A &= 2 \sum_{k=1}^{m-1} k + m(T - m + 1) \\ &= m(m - 1) + m(T - m + 1) = Tm. \end{aligned} \quad (8)$$

这是对式(7)分子的另一种验证。阶梯加平台的这个具体展开采用 $T \geq m$ ；即使 $T < m$ ，按输出计数所得的 $A = Tm$ 仍然成立。

表 3 一维内部块的输入读取缩减倍数

输出边长 T	16	32	64	128	256
$m = 5$	4.0	4.4	4.7	4.9	4.9
$m = 9$	6.0	7.2	8.0	8.5	8.7

将分母写成 $T[1 + (m - 1)/T]$ ，可看出 $R_{ID} = m/[1 + (m - 1)/T]$ 。固定 m 并增大 T ，光环相对于核心区的开销下降，复用逐渐接近 m ；继续扩大分块的收益也随之减小。

3.3 一维边界块：虚拟元素不产生全局读取

考虑最左侧输出块 $P[0]$ 到 $P[7]$ ，仍取 $m = 5, r = 2$ ，并假定右侧有足够多的有效输入。共享内存的 12 个位置对应输入坐标 -2 到 9 ；前两个位置写入零，实际从全局内存读取的只有 $N[0]$ 到 $N[9]$ ，共 10 个元素。

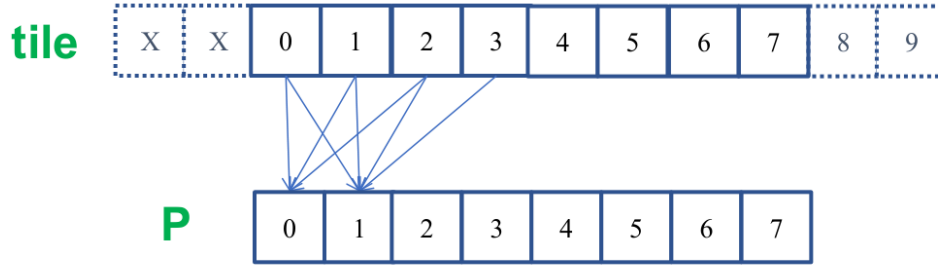


图 4 左边界输入分块中的两个虚拟位置。它们占用共享内存位置，但不需要从输入数组加载。

直接实现计算 $P[0]$ 时只需读取三个有效输入，计算 $P[1]$ 时读取四个，其余六个输出各读取五个。分子和分母都应只计有效的全局读取，因此

$$A = 3 + 4 + 6 \times 5 = 37, \quad B = 8 + 2 = 10, \quad R_{1D,edge} = \frac{37}{10} = 3.7. \quad (9)$$

若乘加循环仍按五个位置执行，分块内核仍可读取 40 次共享内存，只是其中三次读取的是已补入的零。这与“直接实现有 37 次有效全局读取”采用不同的计数对象。

为便于推广，对一侧接触边界、另一侧不接触边界的完整输出块进行形式化计数。设 $T \geq r$ 且输入至少含有 $T + r$ 个元素，最靠边的 r 个输出分别缺少 $r, r-1, \dots, 1$ 个有效输入，故

$$R_{1D,edge} = \frac{Tm - r(r+1)/2}{T+r}. \quad (10)$$

边界例子的比值高于内部例子的 $40/12$ ，原因是低复用的最外侧输入变成了零，既减少直接读取，也减少分块加载。由此得到的是计数比例变化；实际执行还包含补零、边界分支等开销。

3.4 二维内部块：两个方向的复用相乘

二维输出块有 T^2 个输出，每个输出使用 m^2 个输入，直接实现需要 $T^2 m^2$ 次输入读取。不同输入组成边长为 $T + m - 1$ 的完整方形区域，只需加载 $(T + m - 1)^2$ 个元素。因此

$$R_{2D}(T, m) = \frac{T^2 m^2}{(T + m - 1)^2} = \left(\frac{Tm}{T + m - 1} \right)^2. \quad (11)$$

这个平方来自行、列两个方向的几何计数；权重仍可以是任意的稠密 $m \times m$ 矩阵。

当 $T = 8, m = 5$ 时，输入分块为 12×12 ，只需加载 144 个输入；64 个输出共使用 $64 \times 25 = 1600$ 个输入值。于是 $R = 1600/144 \approx 11.1$ ，对应的输入全局读入字节数由 6400 降至 576。这里输出分块大小是 8×8 ，输入分块大小是 12×12 ，二者在分子与分母中承担不同角色。

表 4 二维内部块的输入读取缩减倍数

输出分块	8×8	16×16	32×32	64×64
$m = 5$	11.1	16.0	19.7	22.1
$m = 9$	20.3	36.0	51.8	64.0

固定 m 时， R_{2D} 随 T 增大而趋近 m^2 。较大的权重窗口提供更大的潜在复用，但也扩大光环和计算量。表中的分块大小用于几何分析；能否按“一线程加载一个输入”的策略 2 直接启动，还要检查 $(T + m - 1)^2$ 个线程是否满足设备限制。

3.5 二维边界的乘积计数

二维边界也可先分别统计行、列方向，再相乘。假设一个完整输出块仅在某一侧接触图像边界，且相反侧仍有足够多的输入。边界方向的直接读取计数为 $Tm - r(r + 1)/2$ ，不同输入数为 $T + r$ ；内部方向则分别为 Tm 和 $T + 2r$ 。所以

$$\begin{aligned} R_{2D,edge} &= R_{1D,edge} R_{1D}, \\ R_{2D,corner} &= R_{1D,edge}^2. \end{aligned} \quad (12)$$

角块的两个方向都靠边，边块只有一个方向靠边。以 $T = 8, m = 5$ 为例，三个位置的计数如下。

表 5 同一分块大小在内部、单边与角落处的计数差异

位置	直接读取 A	不同有效输入 B	缩减倍数 R
内部块	$40^2 = 1600$	$12^2 = 144$	11.11
单边块	$37 \times 40 = 1480$	$10 \times 12 = 120$	12.33
角块	$37^2 = 1369$	$10^2 = 100$	13.69

3.5.1 非整块与较小图像的统一写法

当最后一块只有部分有效输出，或图像小到一个分块同时接触两侧边界时，可以直接从有效区间计数。对长度为 D 的一维数组，位置 q 所需的有效邻域长度为

$$\ell_D(q) = \min(D - 1, q + r) - \max(0, q - r) + 1, \quad 0 \leq q < D. \quad (13)$$

有效输出区间为 $[a, b]$ 时，其输入并集长度是 $U_D(a, b) = \min(D - 1, b + r) - \max(0, a - r) + 1$ 。设二维块的有效输出行区间为 $[y_0, y_1]$ ，列区间为 $[x_0, x_1]$ ，则

$$A = \left(\sum_{y=y_0}^{y_1} \ell_H(y) \right) \left(\sum_{x=x_0}^{x_1} \ell_W(x) \right), \quad (14)$$

$$B = U_H(y_0, y_1) U_W(x_0, x_1).$$

这是把每个输出的有效输入数 $\ell_H(y)\ell_W(x)$ 对行列求和后分离所得的计数恒等式。它还说明，应先确定有效输出集合，再统计所需输入；若内核为越界候选输出仍执行乘加，这部分无效算术需要另外记录。

4 带宽、算术强度与计算上限

4.1 未分块卷积的字节 / 浮点操作比

一次卷积乘加读取一个单精度输入，产生一次乘法与一次加法，按两次浮点操作 (floating-point operation, FLOP) 计。输入的 4 字节来自全局内存；权重的 4 字节按常量缓存提供。只统计输入的全局读入量时，每次乘加对应

$$b_0 = \frac{4B}{2\text{FLOP}} = 2B/\text{FLOP}. \quad (15)$$

b 表示字节 / 浮点操作比，数值越大，每完成一定算术工作所需的全局数据越多。它的倒数称为算术强度 (arithmetic intensity)，记为 $I = 1/b$ ，单位为 FLOP/B；未分块的基准值是 $I_0 = 0.5 \text{ FLOP/B}$ 。

对于内部块，若每次输入加载平均支持 R 次使用，算术工作量不变，输入全局读取量降为原来的 $1/R$ ，所以

$$b_{\text{tiled}} = \frac{b_0}{R} = \frac{2}{R} B/\text{FLOP}, \quad I_{\text{tiled}} = \frac{R}{2} \text{FLOP/B}. \quad (16)$$

这些式子连接了几何复用与带宽需求。后续峰值分析沿用这一口径：单精度输入、内部有效输出、权重按常量缓存供给，暂不计输出写回。

4.2 由内存带宽得到吞吐上限

设内存带宽为 $\mathcal{B}$ ，单位为字节每秒；计算峰值为 C_{peak} ，单位为 FLOP 每秒。若每完成一 FLOP 需要 b 字节输入，则带宽最多支持 $\mathcal{B}/b$ FLOP/s。结合算术峰值，模型给出的上限为

$$C_{\text{bound}} = \min\left(C_{\text{peak}}, \frac{\mathcal{B}}{b}\right) = \min(C_{\text{peak}}, \mathcal{B}I). \quad (17)$$

当 $\mathcal{B}/b < C_{\text{peak}}$ 时，内存供给先达到上限；提高数据复用能增加带宽允许的算术吞吐。要使带宽至少能够供给峰值算术，需要

$$b \leq b_{\text{balance}} := \frac{\mathcal{B}}{C_{\text{peak}}}, \quad R \geq R_{\text{min}} := \frac{b_0 C_{\text{peak}}}{\mathcal{B}}. \quad (18)$$

R_{min} 是这一输入流量模型中的必要复用门槛。到达门槛后，线程调度、同步、共享内存访问及其他执行开销仍会影响实际吞吐。

以 $C_{\text{peak}} = 1000 \text{ GFLOP/s}$, $\mathcal{B} = 150 \text{ GB/s}$ 为例, 未分块时带宽上限是 $150/2 = 75 \text{ GFLOP/s}$, 仅为峰值的 7.5%。所需复用为 $1000/75 \approx 13.3$ 。这里 GB 与 GFLOP 都采用十进制数量级, 因此相除时可以直接约去 10^9 。

4.3 不同参数下的复用门槛

将下面三组硬件算例参数代入同一公式, 可以比较计算能力增长与内存带宽增长之间的差异。峰值相对于带宽越大, 每加载一个字节需要承担的算术工作就越多。

表 6 只计输入读取时的带宽上限与复用门槛

算例	C_{peak}	$\mathcal{B}$	未分块上限	峰值比例	$R_{\min}$
约 2010 年 GPU	1000 GFLOP/s	150 GB/s	75 GFLOP/s	7.50%	13.3
2013 年 GRID K520	5000 GFLOP/s	192 GB/s	96 GFLOP/s	1.92%	52.1
H100 PCIe	51 TFLOP/s	2 TB/s	1 TFLOP/s	1.96%	51.0

H100 算例的复用门槛。按所列 51 TFLOP/s、2 TB/s 和 $b_0 = 2 \text{ B/FLOP}$ 计算, $R_{\min} = 2 \times 51/2 = 51$; 表6列出这一参数组合下的计算结果。

一维 $m = 5$ 卷积的内部块复用上限是 5, 二维 $m = 5$ 的上限是 25。即使不断增大分块, 这两种情形也达不到 51 的输入复用门槛。二维 $m = 9, T = 32$ 时, $R = (32 \times 9/40)^2 = 51.84$, 在此模型中刚好超过该门槛。是否能用相应线程组织实现这个分块, 还取决于资源容量。

4.4 权重窗口大小与资源平衡

由式(17)可得, 分块后的模型峰值比例是

$$U(R) = \min\left(1, \frac{\mathcal{B}R}{b_0 C_{\text{peak}}}\right). \quad (19)$$

把 1000 GFLOP/s、150 GB/s 的组合记为模型 A, 把 5000 GFLOP/s、192 GB/s 的组合记为模型 B, 则分别有 $U_A = \min(1, 0.075R)$ 和 $U_B = \min(1, 0.0192R)$ 。下面固定输出分块大小, 改变权重边长。

表中百分比为近似值; 按未舍入公式计算, 二维 $m = 5$ 在模型 B 下为 37.93%, 二维 $m = 9$ 为 99.53%。这些百分比表示带宽模型允许的峰值比例, 不能当作已测得的硬件利用率。

一维的内部块复用趋近 m , 二维则趋近 m^2 , 因此二维卷积用较小的权重边长就可能接近同一个带宽门槛。同时, 扩大权重窗口会增加完成一个输出的算术量; 峰值比例上升, 本身并没有给出处理同一问题所需时间的缩短幅度。

表 7 不同权重窗口对应的近似峰值比例

维度	输出分块	m	模型 A	模型 B
一维	$T = 1024$	5	37%	9.6%
		9	67%	17%
		15	100%	28%
		55	100%	100%
二维	$T \times T = 32 \times 32$	3	60%	15%
		5	100%	37%
		7	100%	67%
		9	100%	接近 100%

4.5 扩展计数：将输出写回计入流量

为了说明计数口径对结论的影响，可以在原有模型上增加输出写回。一个完整二维内部块加载 S^2 个输入，写回 T^2 个输出，进行 $2T^2m^2$ 次浮点操作。因此，只增加输出写回、不改变权重缓存假设时，得到

$$b_{\text{read+write}} = \frac{4S^2 + 4T^2}{2T^2m^2} = \frac{2}{R} + \frac{2}{m^2} \quad \text{B/FLOP.} \quad (20)$$

输入复用可以降低第一项，输出写回项则仍然存在。例如 $T = 16, m = 5$ 时 $R = 16$ ，只计输入得到 $b = 0.125 \text{ B/FLOP}$ ；加上输出后是 $0.125 + 0.08 = 0.205 \text{ B/FLOP}$ 。对应的未分块读写口径为 $2 + 2/m^2$ 。

因此，使用字节数比较实现、或将理论模型与测量结果对照时，应让输入、权重、输出及边界的统计口径保持一致。几何复用分析提供其中最直接的一部分：一次输入加载究竟支持了多少次邻域使用。

5 分块规模与存储层次

5.1 线程数量与共享内存容量

5.1.1 每块的线程预算

策略 2 为每个输入位置配置一个线程，因此每块的线程数、共享内存字节数及输出角色比例分别为

$$n_{\text{threads}} = S^2, \quad q_{\text{shared}} = 4S^2 B, \quad \eta_{\text{compute}} = \frac{T^2}{S^2}. \quad (21)$$

η_{compute} 只表示具有输出计算角色的线程比例。 $T = 8, m = 5$ 时它是 $64/144 \approx 44.4\%$ ； $T = 16, m = 5$ 时为 $256/400 = 64\%$ 。这些线程在加载阶段全部有工作，因而这一比例也不能直接当成整个内核的效率或 SM 占用率。

设设备报告的单块线程上限为 L_{block} ，这种实现必须满足

$$(T + m - 1)^2 \leq L_{\text{block}}, \quad T \leq \lfloor \sqrt{L_{\text{block}}} \rfloor - m + 1. \quad (22)$$

例如，若目标设备的单块线程上限为 1024，则 $m = 5$ 时允许的最大输出边长是 28， $m = 9$ 时是 24。 32×32 输出配 5×5 权重需要 $36 \times 36 = 1296$ 个加载线程，已经超过这一示例上限；配 9×9 权重则需要 1600 个线程。几何表中的 32×32 或 64×64 分块仍可用于分析，但其实实现必须采用不同的线程分工。

由式(11)还可以得到 $R_{2D} = m^2 \eta_{\text{compute}}$ 。这个关系把光环造成的线程角色开销与平均复用联系起来：固定权重边长时，核心区相对输入分块越大，平均复用越接近 m^2 。

5.1.2 每个 SM 的驻留预算

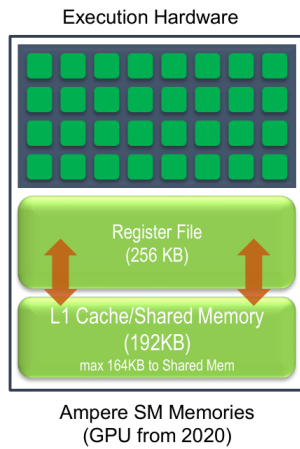
流式多处理器 (streaming multiprocessor, SM) 可以同时容纳多个线程块。单块能否启动与多个块能否同时驻留，分别对应单块限制和 SM 总资源限制。设 SM 的线程容量为 N_{SM} ，共享内存容量为 Q_{SM} ；仅考虑这两项约束时，同时驻留的块数满足

$$n_{\text{resident}} \leq \min\left(\left\lfloor \frac{N_{\text{SM}}}{S^2} \right\rfloor, \left\lfloor \frac{Q_{\text{SM}}}{4S^2} \right\rfloor\right). \quad (23)$$

共享内存容量尚有剩余，并不保证还能增加线程块；线程容量可能已经先被占满。

A100 的 SM 存储示例包含 256 KB 寄存器文件，以及 192 KB 的 L1 缓存 / 共享内存池，其中最多 164 KB 可划作共享内存。存储访问延迟可按粗略层级比较：寄存器约一个周期，共

享内存和缓存 / 常量存储约数个周期，全局内存约数百个周期。这些量级用于解释把反复访问的数据移到片上的动机。



- **Memory access time**
 - registers (~1 cycle)
 - shared memory (~few cycles)
 - cache/constant memory (~few cycles)
 - global memory (~few hundred cycles)
- **Number of thread blocks mapped to an SM is limited by**
 - Total number of threads supported by an SM
 - Amount of shared memory available on SM

图 5 A100 示例中的片上存储容量及粗略访问层次。线程容量与共享内存容量共同限制可驻留线程块的数量。

若每个线程仅搬运一个 4 字节元素，2048 个线程总共只搬运 $2048 \times 4 = 8192$ 字节，约 8 KB。与一个 64 KB 共享内存容量的示例比较，线程数量与可用数据容量之间仍有很大差距。这个容量算式没有指定单块线程上限；实际 `blockDim` 仍需遵守式(22)。

5.2 更大数据块的多元素分工

共享内存中的数据分块可以比物理线程块大。让每个线程分多轮加载输入、承担多个输出，就能用较少的物理线程处理更大的输出分块。此时，线程块大小描述同时参与协作的线程数，数据分块大小描述这一组线程最终准备和处理的数据范围，两者可以分别设计。

一种组织方式是把二维输入分块展平。设共有 n 个线程，线程的线性编号为 t ；让它依次加载 $t, t + n, t + 2n, \dots$ 这些位置，直到覆盖全部 S^2 个输入。加载和补零结束后统一同步，再按类似的循环分配 T^2 个输出。扩大数据分块后，光环占比降低，但每个线程的顺序工作也会增加。

更大分块的带宽模型。对模型 B 和 $m = 7$ ， $T = 32$ 时 $R = (224/38)^2 \approx 34.75$ ，对应峰值比例约 66.7%。取 $T = 80$ ，则

$$S = 86, \quad R = \left(\frac{80 \times 7}{86} \right)^2 \approx 42.40, \quad U_B = 0.0192R \approx 81.4\%. \quad (24)$$

该输入分块占 $86^2 \times 4 = 29584$ 字节。它需要多轮加载与多输出分工，不能直接建立 86×86 个线程。81.4% 是模型上限，尚未包括具体线程布局、同步和执行开销。

5.3 分银行存储与访问组织

寄存器文件和共享内存都分成多个存储体 (bank)，以提供并行的数据通路。多个访问争用同一存储体的服务能力时，可能发生存储体冲突 (bank conflict)，使流水线等待。数据已经位于片上，并不表示任意访问模式都能获得同样的吞吐。

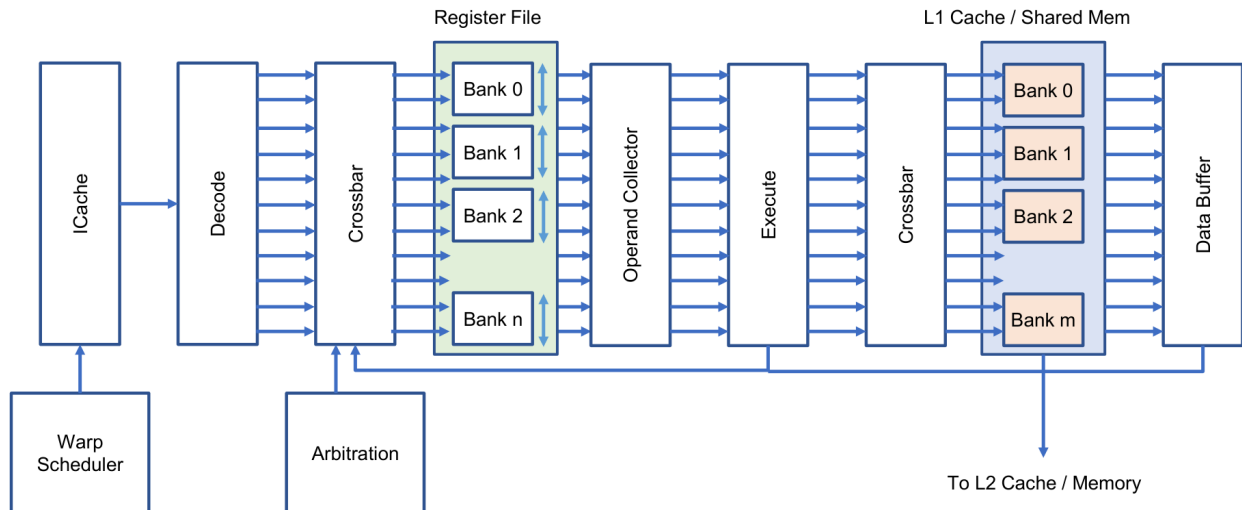


图 6 概念性的 SM 执行流水线。寄存器文件与 L1 / 共享内存均采用多存储体组织，交叉开关连接执行通路、操作数收集与存储通路。

图6中的线程束调度器决定待执行工作，经指令缓存与译码后，寄存器中的操作数通过交叉开关和操作数收集器送到执行单元。涉及存储访问时，数据通路还会经过 L1 / 共享内存一侧；访问更低层存储时连接到 L2 缓存和内存。寄存器与共享内存两处都可能成为数据供给的限制环节。

全局内存则包含多个通道、存储体和页，并依赖成组传输提高效率。合并访存 (memory coalescing) 关注相邻线程的访问如何组合成有效的数据传输。行优先图像中，同一行相邻列的地址连续，所以让相邻的横向线程加载相邻列，有利于形成成组访问。输入读取次数相同的两个实现，也可能因为地址分布不同而产生不同的传输效率。

L1 缓存的非一致性 (non-coherence) 意味着不能仅凭存在缓存，就假设不同缓存副本会随其他副本的写入自动保持一致。数据放置、访问组织和同步分别解决不同的问题。对分块卷积而言，完整的性能分析需要同时看输入复用、线程分工、共享内存访问及全局访问组织；单独提高 R 只改善其中的输入供给部分。

6 例题与关键结论

6.1 角块的全局读取缩减倍数

题目。对 32×32 输入图像使用 5×5 权重和 16×16 输出分块，采用策略 1。求线程块 $(0,0)$ 因共享内存复用而获得的输入全局读取缩减倍数。

输出范围与输入范围。本块负责输出行、列 $0-15$ ，半径 $r = 2$ 。完整输入分块的坐标范围为 $-2-17$ ，边长为 20 ；与有效图像相交后，实际读取的行、列范围都是 $0-17$ 。因此不同有效输入数为 $B = 18 \times 18 = 324$ 。其余 $20^2 - 18^2 = 76$ 个共享内存位置补零。

直接读取计数。输出 $(0,0)$ 需要 $3 \times 3 = 9$ 个有效输入， $(0,1)$ 需要 $3 \times 4 = 12$ 个，第 0 行的其余 14 个输出各需要 $3 \times 5 = 15$ 个。第 1 行有四行有效输入，后面 14 行各有五行有效输入。逐行统计为

$$\begin{aligned} A_0 &= 9 + 12 + 15 \times 14 = 231, \\ A_1 &= 12 + 16 + 20 \times 14 = 308, \\ A_y &= 15 + 20 + 25 \times 14 = 385, \quad 2 \leq y \leq 15, \\ A &= 231 + 308 + 14 \times 385 = 5929. \end{aligned} \tag{25}$$

最终结果为

$$R = \frac{5929}{324} \approx 18.30. \tag{26}$$

乘积验证。单个方向的有效邻域长度之和为 $3 + 4 + 14 \times 5 = 77$ ，二维直接读取数就是 $77^2 = 5929$ 。这与式(14)及逐行展开一致。采用策略 1 影响的是 256 个线程如何分多步装入这些输入；只要完整输入被正确装入且每个有效元素只读一次，计数结果仍由 324 个不同有效输入决定。

把本题当成内部块，会得到 $16^2 \times 25/20^2 = 16$ 。这一结果对应的是“ 400 个输入位置全部有效”的另一种位置条件。本题角块应同时排除直接实现中的越界读取和分块加载中的虚拟输入，得到 18.30 。

6.2 输入型线程块中的控制分歧

题目。二维输出分块为 30×30 ，权重为 3×3 ，输入分块为 32×32 。采用策略 2，计算输出时每个线程块中有多少个线程束发生控制分歧？

线程束与行的对应。这里每个线程束含 32 个线程。二维块按 x 坐标优先展平，线性编号为 $t = t_x + 32t_y$ ，线程束编号是 $w = \lfloor t/32 \rfloor = t_y$ 。因此每一行恰好构成一个线程束，总计 $1024/32 = 32$ 个线程束。

逐线程束检查。输出角色条件为 $t_y < 30$ 且 $t_x < 30$ 。对于 $w = 0-29$ ，同一线程束中 $t_x = 0-29$ 的 30 个线程进入乘加循环， $t_x = 30, 31$ 的两个线程不进入，故这 30 个线程束都有控制分歧。对于 $w = 30, 31$ ，所有线程都不满足行条件，整个线程束统一跳过计算。

表 8 计算角色判断在各线程束中的结果

线程束编号	线程束数量	每束计算线程	每束不计算线程	是否分歧
0-29	30	30	2	是
30-31	2	0	32	否

因此，有 30 个线程束发生计算角色分歧，2 个线程束没有该分歧。全部线程统一不执行某个分支，仍然是统一的控制流。所有 32×32 个线程都参与输入分块的加载或补零；上述答案只统计计算角色条件，图像边缘的加载和写回判断应按实际位置另行检查。

当输入块的边长 S 不等于 32 时，一个线程束可能跨越多行。这时应先用 $t_x = t \bmod S$ 、 $t_y = \lfloor t/S \rfloor$ 恢复局部坐标，再检查每组线程的条件取值，不能继续把一行直接当成一个线程束。

6.3 A40 算例中的带宽平衡与复用门槛

题目。采用 A40 算例给出的 FP32 峰值 37.4 TFLOP/s、内存容量 48 GB 和带宽 696 GB/s。求充分供给计算峰值所需的字节 / FLOP 比，并说明卷积输入需要多少复用。

平衡点的两种单位。这一比值由带宽与计算峰值决定，内存容量不参与计算。统一单位后，可得

$$\begin{aligned}
 b_{\text{balance}} &= \frac{696}{37400} \approx 0.0186096 \text{ B/FLOP} \approx 0.019 \text{ B/FLOP}, \\
 I_{\text{balance}} &= \frac{37400}{696} \approx 53.7356 \text{ FLOP/B}.
 \end{aligned} \tag{27}$$

也就是每从全局内存取得一个字节，需要支持约 53.7 次浮点操作，才与这一峰值计算 / 带宽比平衡。

输入复用门槛。上述 53.7 的单位是 FLOP/B。要得到相对于未分块卷积的输入复用倍数，还要使用其基准 $b_0 = 2 \text{ B/FLOP}$ ，或等价的 $I_0 = 0.5 \text{ FLOP/B}$ ：

$$R_{\min} = \frac{b_0}{b_{\text{balance}}} = \frac{I_{\text{balance}}}{I_0} = \frac{2 \times 37400}{696} \approx 107.47. \quad (28)$$

从单个输入元素看也一样：4 字节需要支持约 $4 \times 53.7356 = 214.94 \text{ FLOP}$ ，而一个输入的一次卷积使用贡献两次 FLOP，故需要约 $214.94/2 = 107.47$ 次平均使用。推算时保留未舍入的带宽比，可以避免过早使用 0.019 带来的数值偏差。

三个量的区别

0.01861 B/FLOP 是峰值平衡所允许的字节 / 操作比；53.74 FLOP/B 是它的倒数；107.47 是在未分块卷积基准之上所需的无量纲输入复用倍数。前两个量由设备算例的带宽与峰值给出，第三个量还依赖算法的基准数据使用方式。

6.4 关键公式与实现关系

二维分块卷积的实现与分析可以按同一条依赖关系组织：先确定输出分块，向外扩展出输入分块，再把输入位置分配给线程，最后统计这些加载支撑的有效计算。表9汇总了其中的关键关系。

表 9 几何、线程、复用与吞吐之间的关系

问题	核心关系	适用条件或含义
输入分块	$S = T + m - 1 = T + 2r$	输出每侧扩展半径 r 。
块与网格	块为 $S \times S$ ；网格按 $[W/T] \times [H/T]$	策略 2；输出步长为 T 。
输入坐标	$y_i = y_o - r, x_i = x_o - r$	所有线程分别加载或补零。
输出计算	$t_y < T, t_x < T$ ；访问 $\text{tile}[t_y + i][t_x + j]$	同步后计算；写回还需输出边界判断。
内部块复用	$R_{1D} = Tm/S$ ； $R_{2D} = (Tm/S)^2$	完整加载；计输入全局读取。
边界块复用	$R = A/B$ ， A, B 均只计有效输入	虚拟位置补零，不产生输入全局读取。
带宽模型	$b = 2/R$ ； $I = R/2$	单精度乘加；只计输入读取。
计算平衡	$R_{\min} = b_0 C_{\text{peak}}/B$	模型中的必要复用门槛。

核对代码时，输入边界、全块同步、计算角色和输出边界应依次成立；行坐标与高度比较，列坐标与宽度比较。核对计数时，先区分输入分块的存储位置数、真正发生的全局加载数，以及所有输出累计的输入使用次数。核对性能结论时，则将几何上的复用、资源允许的线程布局和带宽模型的上限分别算清，再建立它们之间的关系。